



Orientação a Objetos



Interfaces



Professor

Me. Jonas Pontes



@jn.pontes

Considerações sobre herança



1 A herança pode comprometer o encapsulamento, pois os detalhes internos da classe pai tornam-se visíveis e acessíveis nas classes filhas.



Considerações sobre herança



2

A herança aumenta o acoplamento entre classes, dificultando a manutenção do código. Qualquer mudança na classe pai pode impactar diretamente todas as suas subclasses.



Considerações sobre herança



3 Com herança, não é possível modificar o comportamento herdado em tempo de execução, o que limita a flexibilidade do sistema.



Considerações sobre herança



Use herança se...

- A classe filha expressar "um tipo especial de" e não "ser um papel desempenhado por";
- Uma instância de uma classe filha nunca precisar tornar-se um objeto de outra classe;



Considerações sobre herança



Use herança se...

- A classe filha estender ao invés de substituir total ou parcialmente as responsabilidades da classe mãe;
- A sua hierarquia de herança representar um relacionamento "é um" e não um relacionamento "tem um";



Considerações sobre herança



Use herança se...

- Você desejar ou precisar realizar alterações globais para as suas classes filhas alterando uma classe mãe;
- Você precisar aplicar a mesma classe e métodos a diferentes tipos de dados;





Interfaces



Retornemos ao projeto empresa



Considere uma classe chamada
`SistemaInterno`;

Imagine que essa classe tem um método
de login, o qual recebe um funcionário;

O corpo desse método deve invocar o
método `autenticar`;



Retornemos ao projeto empresa



Mas nem todo funcionário tem o método autenticar;

Uma solução é o método login receber um gerente, que tem o método autenticar;
O que fazer se outro funcionário, como vendedor, também precisar se autenticar?



Retornemos ao projeto empresa



A classe `SistemaInterno` pode ter dois métodos `login`, um que recebe um gerente e outro que recebe vendedor.

Problema de design: a cada nova subclasse de `Funcionario` que seja autenticável implica criar outro método `login`.



Retornemos ao projeto empresa



Uma solução é criar uma nova classe (`FuncionarioAutenticavel`) que seja subclasse de `Funcionario` e superclasse de `Gerente`, `Vendedor` e qualquer outro funcionário autenticável.

Essa classe deve ser classe abstrata



Retornemos ao projeto empresa



Resolvido?

Por enquanto sim. Todavia...



Retornemos ao projeto empresa



O sistema agora tem uma classe chamada Fornecedor;

Os fornecedores precisam se autenticar ao sistema, por meio do SistemaInterno.

O que fazer?



Retornemos ao projeto empresa



- 1 Sobrecarregar o método login?
- 2 Fazer Fornecedor ser subclasse de FuncionarioAutenticavel?



Retornemos ao projeto empresa



1 é problemático pelo motivo que já vimos (possibilidade de excesso de sobrecarga);

2 é uma herança sem sentido: um Vendedor teria um método de bonificar, ter salário etc!



Retornemos ao projeto empresa



Uma solução é usar interfaces



Interface



Interface é uma especificação, um contrato que define que cada classe que aderir a ela se compromete a implementar métodos na interface declarados.



Interface



Toda classe define dois itens:

- O que uma classe faz (as assinaturas dos métodos);
- Como uma classe faz essas tarefas (o corpo dos métodos e atributos privados).



Interface



Pode-se, então, criar um "contrato" o qual define tudo o que uma classe deve fazer se quiser ter um determinado status.

Quem quiser ser autenticavel precisa saber autenticar uma senha, devolvendo um booleano



Interface



- Especifica quais operações são permitidas, mas não como elas são realizadas.
- Descreve um conjunto de métodos que podem ser chamados em um objeto.



Interface



- Uma declaração de interface começa com a palavra-chave `interface`;
- Todos os membros de interface devem ser `public` e as interfaces não podem especificar nenhum detalhe de implementação;



Interface



- Todos os métodos declarados em uma interface são implicitamente métodos `public abstract`;
- Para utilizar uma interface, uma classe concreta deve especificar que ela implementa a interface e declarar cada método de interface com a assinatura especificada na declaração de interface.



Interface



- Uma classe abstrata pode não implementar todos os métodos da interface.
- Implementar uma interface é como assinar um contrato com o compilador que afirma: “Irei declarar todos os métodos especificados pela interface” ou “irei declarar minha classe abstract”.



Interface



- Interface é utilizada quando classes díspares precisam compartilhar métodos e constantes comuns;
- Isso permite que objetos de classes não relacionadas sejam processados polimorficamente, pois objetos de classes que implementam a *mesma* interface podem responder às mesmas chamadas de método.



Interface



- Uma interface é geralmente utilizada em lugar de uma classe abstrata quando não há implementação padrão a herdar, isto é, nenhuma variável de instância e nenhuma implementação de método padrão.



Interface



- Uma classe pode implementar várias interfaces;
- Interfaces não tem estado;
- Interfaces podem ter métodos com corpo, desde que seja colocado a palavra default antes do tipo de retorno do método.

